



并行与图结构

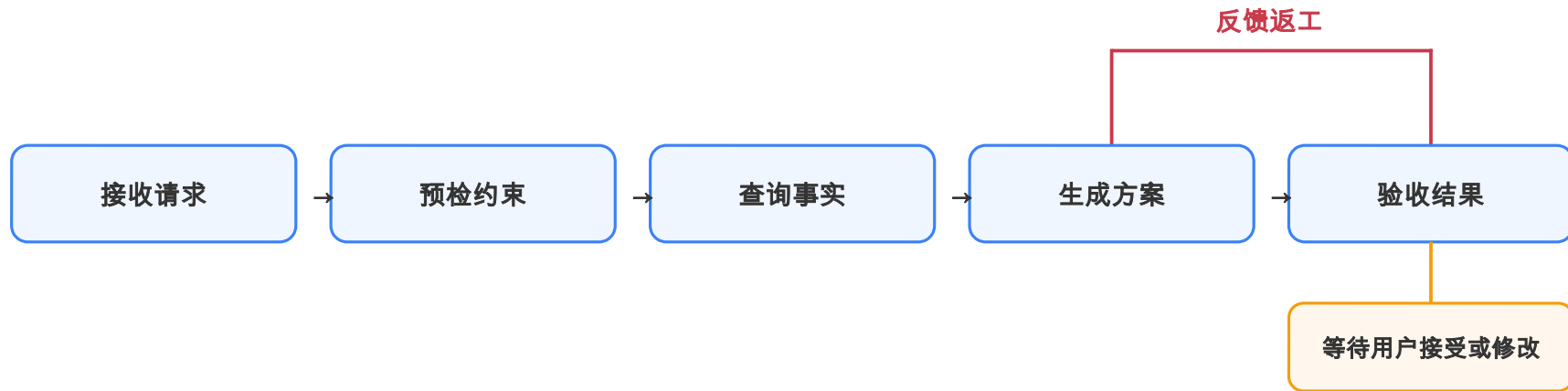
Loop 是 Graph 的特例

Graph

DK

从 Loop 到 Graph : 什么时候该把控制流展开

## 把 *Loop* 压成一条主线



验收发现问题就反馈回生成返工；通过则停下等用户接受或修改——一个 *while* 串起整条主线

这就是 *Loop* 的执行主线：看观察、修动作，一轮轮推到停止条件

# Loop 与 Graph 不是替换关系

## Loop · 环形公交线

车绕着圈跑，简单清楚；但画不出哪几条能同时走

## Graph · 城市交通图

环线 + 分叉 + 汇合 + 换乘；能画出并行与等待

Loop 答不上：哪些步骤能同时干？失败只需重做哪一段？走到哪儿该停下来等人？

Graph 不替代 Prompt / Context / Harness——只把藏在循环里的控制流拎出来，变成可声明的节点和边

那什么时候真该上 Graph？下一页用五个信号判断升级时机

## 什么时候值得从 Loop 演化到 Graph · 五个信号

1

独立的耗时任务本可并行

多个输入互不依赖，却被排成一队串行等待

2

分支多到 if-else 走不动

条件越加越多，代码已经难以走读

3

不同失败要不同返工范围

改文案不该重查事实，返工范围需要精确控制

4

流程需要停下来等人

等待用户、等待审批、等待材料

5

团队要看见并测试结构

需要看见节点、路径，并对图结构做断言

这不是打分表：短、直、单线程的流程，继续用清晰的 *while* 反而更简单

## 收益和代价，必须一起入账

### 图化的收益

- 依赖写成边 → 执行器能识别并行前沿
- 回边落在哪 → 返工范围就停在哪
- 等待成为节点 → 中断与恢复有明确位置
- 一份声明 → 同时驱动执行、测试、可视化

### 要一起承担的代价

控制流变成数据后，团队要设计：

共享状态 / 增量合并 / 汇合条件 /

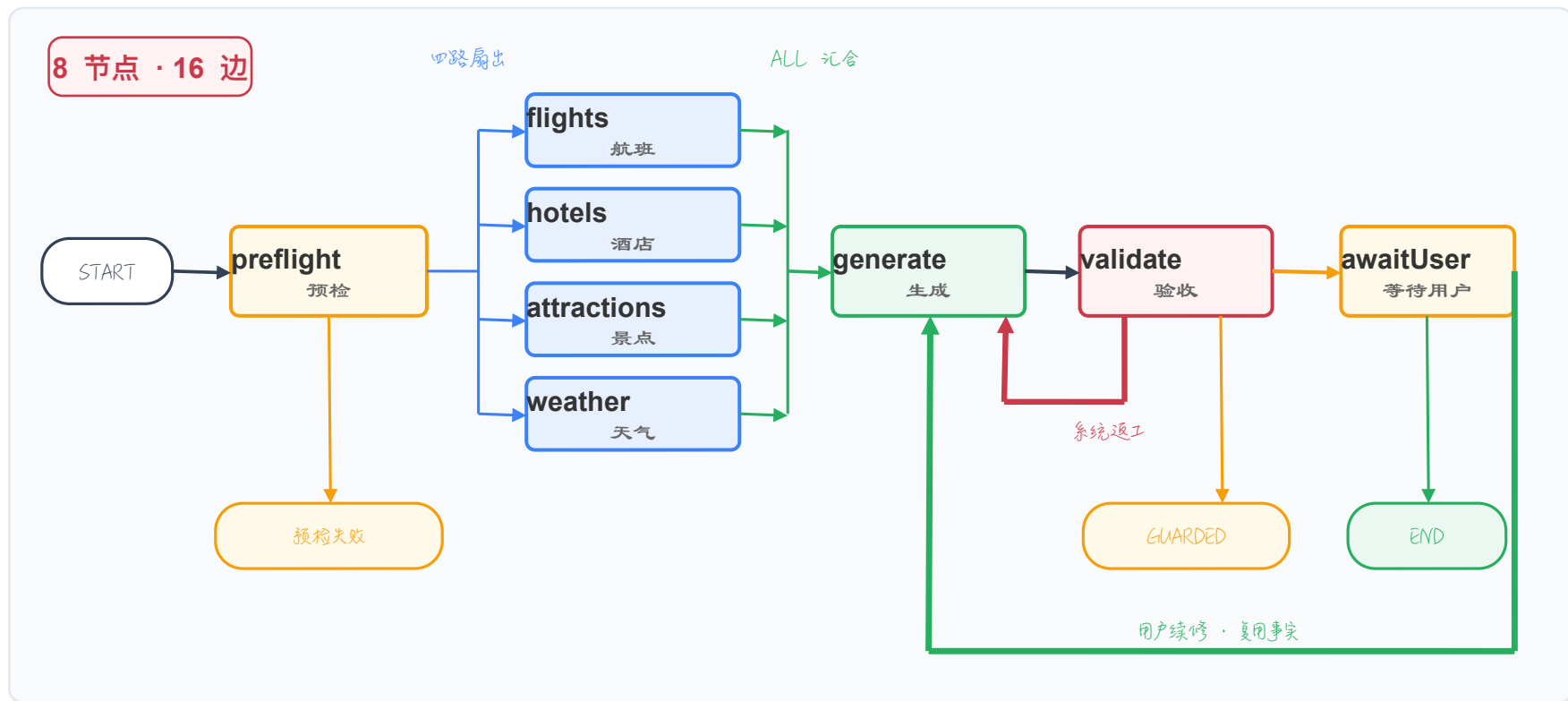
超时 / 取消 / 迟到结果。

合同没写清，只是把复杂度搬进执行器

就像把单行道改成立交桥——通行能力提高了，匝道、信号和事故处理也要一起设计

三问：依赖是否更清楚？返工范围是否更小？团队是否愿担治理成本？

## LoopTrip 的最终目标图 · 8 节点 16 边



四路扇出 → ALL 汇合 → 系统返工 → 用户续修：本章会让这张图分三步长出来

## 顺着箭头走一遍目标图 · 六步读懂执行路径

主线：入口 → 预检 → 四路查询 → ALL 汇合 → 生成 → 验收 → 等待用户

### 01 入口与预检

START → preflight

检查日期、目的地、预算、人数  
不合格：直接结束，不调用工具

### 02 四路并行扇出

preflight → 4 facts

航班 / 酒店 / 景点 / 天气  
共享 *PlanRequest*，可以同时执行

### 03 ALL 汇合

4 facts → generate

每路交回结果或结构化降级  
四份到齐，*generate* 只启动一次

### 04 生成与验收

generate → validate

验收失败且还有轮次：只回 *generate*  
复用事实，不再重跑四路查询

### 05 守住不可交付

validate → GUARDED

必需数据缺失，或返工轮次耗尽  
主动进入受保护结束

### 06 等待用户决定

validate → awaitUser

满意 → END；续修 → *generate*  
日期或目的地变化：作为新请求

**并行的边界：**四路只读同一份 *PlanRequest* 才天生并行；若酒店要读航班到达时间，补一条 *flights*→*hotels* 边即转串行——图上的边说了算

先走主线，再看三处分流：预检失败直接结束；验收失败局部返工；用户续修复用事实



## 看懂目标图，只问四个问题

1

每个节点负责什么

各做一段明确工作

*preflight* 检

查、*generate* 生

成、*validate* 验收

2

每条边何时能走

边负责指出下一步

条件写在箭头上

3

汇合时要等谁

四个事实节点全部交回

结果

*generate* 才启动一次

4

结束是哪种结局

*END* 正常完成 /

*GUARDED* 守住边界 /

预检失败：请求不能执

行

图能直接回答这四个问题，控制流才算真正展开

下一节进入代码层——而且先从三个节点开始，证明主干跑对再加并行和等待

## 本节小结

*Graph* 不是替换 *Loop*，而是把循环、分支、汇合和等待一起展开。

*LoopTrip* 已同时出现独立查询、条件分支、局部返工和用户等待——到了图化时刻。

### 回到主线

五个升级信号判断是否图化；最终方向是 8 个业务节点、16 条边、一个四源 ALL 汇合。

### 你带走的能力

会判断 " 什么时候该把控制流展开 "，并能同时说清图化的收益与要承担的治理代价。

换了走法，但没换结果

把 while 翻译成节点和边：怎么证明没改坏？

## 一笔 " 杭州三天 " 请求，藏着三类问题

“ 从上海出发，帮我安排杭州三日游，预算 3000 元，10 月 1 日出发。 ”

1

这件事能不能做？

目的地、日期、人数、预算齐不齐；  
时间合不合理；约束冲不冲突。信息  
不全就该在入口拦下。

2

怎样产出一版方案？

把航班、酒店、景点、天气等事实查  
回来，交给模型加工成一份可讨论的  
候选行程。

3

这版方案能不能交？

检查结构、预算与规则，判断能交付、  
要返工，还是必须守住边界停下。

Loop 变 Graph，不是 while 不能用，而是业务路径已经值得被看见

这一节要做的，就是把原来藏在循环里的三段工作和几种出口，显式摆到图上

## 同一张图，分三步长大

### STEP 01

先做等价迁移

3 节点 · 7 条边

这一阶段只解决

只改控制流写法，不改业务行为

结构变化

`preflight / generate / validate`

把 `while` 判断拆成有名字的边

验收：原有回归测试全部通过

### STEP 02

再打开并行

7 节点 · 14 条边

这一阶段只解决

缩短等待，同时守住失败隔离

结构变化

航班 / 酒店 / 景点 / 天气

四路事实独立执行，ALL 汇合

验收：总耗时接近最慢一项

### STEP 03

最后加入暂停恢复

8 节点 · 16 条边

这一阶段只解决

让真人确认成为国内一等控制点

结构变化

新增 `awaitUser` 与服务端快照

内环求合格，外环求合意

验收：恢复不重查事实、重启结果一致

推进原则：一次只增加一种不确定性——出错时才能立刻判断是迁移错、并行乱，还是恢复漏。

课堂推演只聚焦当前阶段的图：先看清路径，再增加并行，最后加入暂停恢复

## 第一步：三个节点，就是三个业务问题

①

preflight · 能不能开工

检查目的地、日期、人数、预算是否完整，时间是否合理，约束是否冲突。  
缺条件就在入口说明，不再往下查酒店、调模型。

②

generate · 产出一版方案

当前阶段航班、酒店、景点、天气四类事实仍在此串行查回，再把事实与约束交给模型，加工成候选行程。

③

validate · 能不能交付

检查结构契约、预算与 C1 - C7 规则。  
通过 → END；还有机会 → 回 generate；无法继续 → GUARDED。

preflight



generate



validate



缺条件：PREFLIGHT\_FAILED

加上入口边 START → preflight，正好七条边

✓ 通过：END

↔ 可改：回 generate

✗ 不可继续：GUARDED

validate → generate 不重做预检；当前事实仍在 generate 内查询，下一节拆成事实节点后再保证返工不重查

## 旧职责没消失，只是都挂了新牌子

1

入口检查

→ *preflight* 节点。火星能不能去？这个窗口入口就回答，不再往下查酒店、调模型。

2

验收出口

→ *validate* 的三条边。缺返程？沿返工边回去重写，不重做入口预检。

3

取簿检查

→ *generate* 入口的护栏。本轮中途按下停止，也要走完本轮入账才停，不留半截账。

4

记忆项

→ *GraphState* 工单。第 1 轮缺了什么、第 2 轮补了什么，翻开全在。

两种“再来一次”别记成一笔账：模型偶发失败 = 技术重试；方案设过验收 = 业务返工

技术重试不增加业务轮次；只有 *validate*→*generate* 才开启下一轮业务修改

## 这四样牌子，在代码里叫什么？

1

节点 = NodeSpec

工位 在代码里的学名。一个窗口只办一件事：

*preflight* 只查开工条件，不写行程。

2

边 = EdgeSpec

路牌的学名。它自己不干活，只回答一句话：这一步做完，该去哪一站。

3

终态 = 结果章

最后盖的章只有三枚：通过 *END*、守边界

*GUARDED*、入口不成立 *PREFLIGHT\_FAILED*。

拿到“通过”才算真的交付。

4

工单 = GraphState

随身本的学名。窗口不在公共工单上乱写，只交回自己那一步的小条，由执行器统一贴上去。

节点是工位，边是路牌，终态是结果章，工单是随身本

先认齐这四样再去看代码，*builder* 链里就不会迷路



## 三条业务路径：正常、返工、入口失败

### 三条固定业务轨迹

#### 正常

START → preflight → generate → validate → END

#### 返工

START → preflight → generate → validate  
└──────────┘ 需要修改  
→ 再验收 → END

#### 入口失败

START → preflight → PREFLIGHT\_FAILED  
(模型调用次数 = 0)

返工只回 generate：不重做 preflight；当前事实查询仍在 generate 内，下一节拆成独立事实节点后才保证返工不重查。

## Prompt U · 把现有 runLoop 等价迁入 LightGraph

Prompt U (节选)

这轮只改变控制流的表达方式。把现有 runLoop 拆成 preflight、generate、validate 三个业务节点；本轮只交付、只讲解三节点 7 条边。事实查询先留在 generate 内串行执行，不做并行，也不实现 awaitUser。

动代码前，先用一张对照表告诉我：旧 runLoop 的预检、技术重试、C1-C7、反馈、bestRound、轮次、取消、护栏、事件和终态放到哪；再列出准备修改的文件和测试。等我确认后再实现。

实现 NodeSpec、EdgeSpec、GraphDefinition 和领域无关的 LightGraph。  
节点读不可变 GraphState，只返回 StateDelta；maxSteps 先设 25。  
先用玩具图验证语义，再验证当前图是 3 节点 7 边，并跑完整回归。

## GraphDefinition · 把路线写成可检查的数据

TravelPlanningEngine.java

```
return GraphDefinition.<GraphState>builder()
    .start("START")
    .node("preflight", this::runPreflightNode)
    .node("generate", state -> runGenerateNode(state, events))
    .node("validate", state -> runValidateNode(state, events))
    .terminals("END", "GUARDED", "PREFLIGHT_FAILED")
    .edge("START", "preflight", "always")
    .edge("preflight", "generate", "feasible")
    .edge("preflight", "PREFLIGHT_FAILED", "infeasible")
    .edge("generate", "validate", "always")
    .edge("validate", "END", "accepted")
    .edge("validate", "generate", "needs_revision")
    .edge("validate", "GUARDED", "cannot_deliver")
    .build();
```

真实 API : *builder* → 逐个 *node* → *terminals* → 7 条 *edge* → *build* ; 路线构建后冻结, 运行中不能改线

## LightGraph · 核心三步 + 两道护栏

①

执行当前节点

找到当前节点并运行，拿到它交回的 *StateDelta*——"我这一步改了什么"。

②

把结果写进状态

*state.apply(result.delta())*：节点从不乱写公共工单，由执行器统一合入 *GraphState*。

③

按新状态选下一条边

*definition.next(...)*：一定要在合并之后，否则读到的还是上一轮结果，可能走错门。

两道护栏：业务轮次入口检查取消（本轮走完才停）； *maxSteps = 25* 防止图无限兜圈。

"先合并，再选边"——系统永远依据刚完成的业务结果决定下一步

循环没消失，只是"下一步去哪"不再写死在 *while*，而由路线图声明

## 现场验证 · 换了走法，不能换掉结果

界面看不出 Loop 变 Graph：`plan()` 签名与事件不变，等价迁移的结构证据只在测试里；界面差别要等下一节并行提速、第四节走到 `awaitUser` 停下。

1

先跑课程脚本，讲清三条业务轨迹

`./scripts/course/ch09-s2-graph-migration.sh` — 一次通过：`END`/1 轮；返工一次：多走 `validate`→`generate`→`validate`；入口失败：`PREFLIGHT_FAILED`/ 模型调用 0 次

2

再跑完整回归，确认旧行为未变

`mvn test` — 第七、八章旧测试与原有断言不改；关键轨迹、终态、业务轮次和模型调用次数都要对上基线（录制环境用 `Java 21`，与项目构建同基线）

输入相同、关键轨迹相同、结果相同，旧测试不改也能过——这才叫等价翻译

只要有一条对不上（返工多记一轮 / 预检失败调了模型），就是改写，必须回头查

## 本节小结

一笔旅行请求本来就要经历 " 能不能做、做出方案、能不能交 "，还会有通过、返工、拒绝。

我们只是把它从 *while* 里搬到 *preflight / generate / validate* 三节点 7 边，业务一步没变。

### 回到主线

*runLoop* 等价迁入 *LightGraph*；控制流从 *while* 换成图声明，用玩具图与真实回归证明等价。

### 你带走的能力

会把一段 *while* 等价翻译成可声明的节点和边，并用 " 先合并再选边 " 守住循环语义。

并行从图里长出来：四个事实节点，从相加到取最大

## 方案还没开始写，用户为什么已经等很久

不是模型慢，而是模型动笔前，系统先查航班、再查酒店、再查景点、最后查天气——**排队执行**，用户先把四段等待付了一遍。

?

查天气要等酒店吗

不需要。天气查询不使用酒店的结果。

?

查景点要先拿航班吗

不需要。景点查询不依赖航班结果。

=

四项只依赖同一份请求

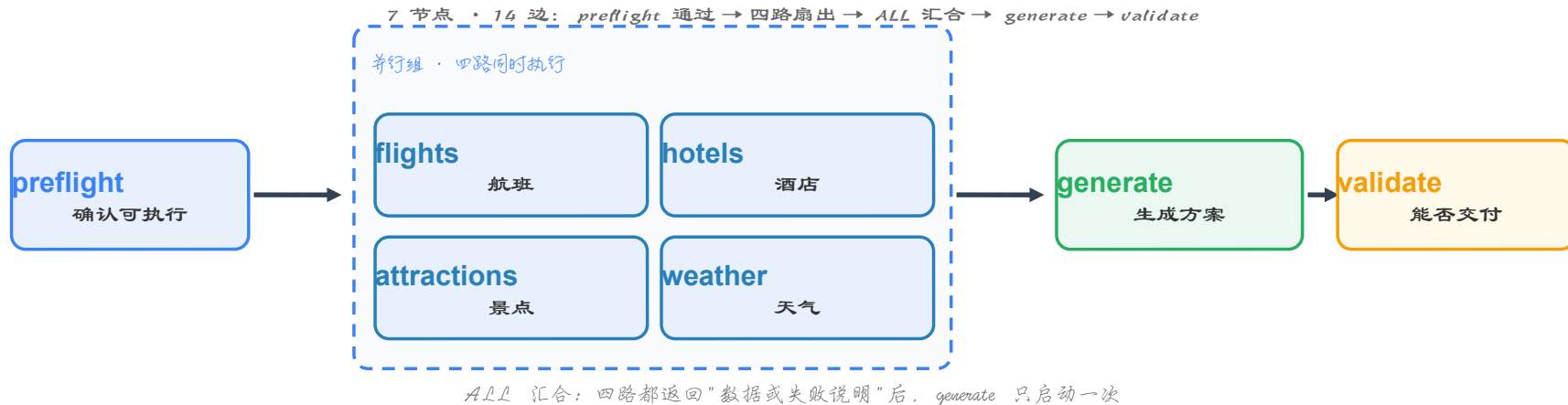
业务上谁也不等谁，执行时就没必要排队。

第一个目标：让用户只等最慢一项，不再等四项相加；第二个目标：更快，但不牺牲交付底线

天气失败不能冲掉另外三项；航班失败也不能让模型凭空编一个再标成功



四路同时出发，收齐“数据或失败说明”再生成



```
JoinSpec.java ·  
TravelGraphFactory.java  
  
public record JoinSpec(String targetNode, Set<String> sourceNodes) {}  
  
.join("generate", "flights", "hotels", "attractions", "weather")
```

*join* 组只挂 *generate* 这四条入边; *validate* → *generate* 返工边不在组内——返工不重查事实、不触发并行

## 为什么更快 · 从 "四项相加" 到 "只等最慢一项"

串行执行 · 时间相加

一件接一件排队完成：

$$1.6 + 1.2 + 1.4 + 0.9$$

$$\approx 5.1 \text{ 秒}$$

模型还没动笔，四段等待已付清

并行执行 · 只等最慢

四项从同一起点出发：

$$\text{MAX}(1.6, 1.2, 1.4, 0.9)$$

$$\approx 1.6 \text{ 秒}$$

只需等最后一个人回来

关键不是 "四个线程"，而是 "这四件事互不依赖"——依赖一变，执行顺序也必须跟着变

并行前沿：现在谁都不用等谁，就让谁一起出发（若景点必须先知道天气，就不能同时查）

## 两类失败，两种处理：故障隔离 VS 质量裁决

航班、酒店 = 核心承诺

失败后：幂等重试 1 次；仍失败则保留缺口。

可以继续进 *generate*，产出诚实的缺口说明，  
但 *validate* 必须阻止交付。

最终结果：进入 *GUARDED*，模型不许猜着补。

景点、天气 = 增强信息

失败后：记录失败原因并降级。

核心行程仍可能成立，  
可以带着明确警告继续交付。

最终结果：其他检查通过时可带警告 *SUCCESS*。

JoinSpec 只管“人是否都回来了”；*validate* 才管“手里的东西够不够交付”

继续执行是故障隔离，允许交付是质量裁决——两件事，两个层面

## 超时的原则：到点收口，迟到不采用

### 超时

单节点 5s · 整组 10s

组开始固定不可变 *deadline*，每个结果记完成时刻。

只有完成时间  $\leq$  *deadline* 才允许合入 *GraphState*。

### 重试

仅限幂等读操作

航班 / 酒店重试 1 次：同条件再查不会多出订单。

红线：接入预订 / 支付后禁止照抄——写操作要带幂等键或交人工确认。

登机口到点关闭——乘客后来赶到，不代表航班重新开门；正确性押在“迟到坚决不采用”

两条边界带走：读操作才能安全重试；*cancel(true)* 只是尽力通知，任务可能停不下来，但迟到结果一定不能进状态。

## Prompt V · 只让 AI 增加并行，不改业务规则

### Prompt V (节选)

基于现有 `GraphDefinition` 和 `LightGraph` 做增量改造：

1. `flights/hotels/attractions/weather` 从 `generate` 拆成四个独立节点；
2. 四节点只读同一份 `PlanRequest`，可同时执行；保持 7 节点 14 边；
3. 增加四源 `JoinSpec`；四项返回数据或明确降级后，`generate` 只启动一次；
4. `validate-generate` 返工只重写方案，必须复用已有事实；
5. `flights/hotels` 必需，幂等重试 1 次仍失败只能 `GUARDED`；
6. `attractions/weather` 增强，失败保留原因，可带警告交付；
7. 单节点 5s、整组 10s；取消或迟到结果不得写入 `GraphState`；
8. 并行任务使用有界的独立 `tool-pool`，不阻塞 `planning-pool`；
9. 四路结果组成 `PlanningFacts` 快照，由 `ContextAssembler` 注入上下文；
10. 保留原有 `@Tool`；上下文已提供快照，提示模型优先使用。

先说明依赖、`Join` 等待项和失败矩阵，确认后再实现。

## 核心代码 · 选边之后只有两条路

```
LightGraph.java

// 选边之后只有两条路 ( LightGraph.run L64-L79 )
List<String> targets = definition.nextTargets(current, state);
Optional<JoinSpec> join = definition.joinForSources(targets);
if (join.isPresent()) {
    ParallelRun<S> parallel = runParallelGroup(
        definition, targets, state, guards, join.get().targetNode());
    state = parallel.state();
    current = join.get().targetNode();
} else {
    current = targets.get(0);
}
// 命中→四路并行, 收齐只启动一次 generate; 未命中→单线, 返工边正是这一支
```

命中交卷名单：四路同时出发、只合并按时交回的、直奔 *generate* 只启动一次；未命中：单线走——返工边  
*validate*→*generate* 正是这一支，不重查事实。

## 效果演示 · 用真实请求看四路怎样汇合

默认请求：2026-10-01 上海→杭州 · 3天 · 总预算 3000 · 酒店 700/晚 · 必去西湖/灵隐寺 · 不要太赶

1

跑真实主流程，看四路先并行

*preflight* 通过后，*flights* / *hotels* / *attractions* / *weather* 进入同一批；完成顺序与耗时以现场为准

2

四路收齐后，*generate* 才启动

事件流只证明组间顺序；真实模型可能一轮通过、返工或进入护栏，终态以现场为准

3

若发生返工，只回 *generate*

核对 *validate*→*generate* 后四个事实节点不再执行；天气超时、航班失败、迟到隔离交给自动化测试

现场只证明两件事：先并行、收齐后生成；返工不重复查询事实

## 本节小结

并行不是 " 先开线程 "，而是 " 先看依赖 "。依赖上谁都不等谁，执行上就没理由排队。

把四个事实查询拆成节点，图从 3 节点长成 7 节点 14 边，一个四源 ALL 汇合。

### 回到主线

`JoinSpec` 让 `generate` 等齐四源只触发一次；降级是返回值不是异常；迟到结果坚决不采用。

### 你带走的能力

会用图声明识别并行前沿，并把并行、故障隔离、交付边界这三件事分层做对。



图上的暂停键——中断节点与双环一张图

## 用户还没决定，流程不能假装结束

一份 " 杭州三日游 " 通过机器验收后，用户可能说 " 满意 "，也可能说 " 第二天轻松点 "，还可能改成 " 下周去上海 "。



后厨已完成质检

菜做好端到桌边，机器验收通过——  
但这只是 " 合格 "。



客人还没表态

在用户开口前，系统既不能说失败，  
也不能擅自说完成。



等待是正式状态

这段等待不是出错，而是业务流程里  
的一个站点。

给图增加一个节点 `awaitUser` : `validate` 通过后不再直接走 `END`，而是先停在这里

这一节先解决：用户没决定时图停在哪里；用户开口后流程从哪里继续

## 用户开口前 · 三种回答，三条路

方案通过 `validate` 后停在 `awaitUser`：先想清用户会怎么回答，再决定这张图往哪走。

①

「满意，就这样」

用户认可当前方案。这是唯一走向 `END / SUCCESS` 的出口——机器判合格之后，还要人判合意，流程才真正结束。

②

「第二天轻松一点」

目的地、日期、查询条件都没变 → 复用已查到的航班、酒店、景点、天气，只回 `generate` 重写方案，再交 `validate` 验收。

③

「改成下周去上海」

日期或目的地变了，旧事实全部失效 → 不临时加回边硬绕，结束当前链路，作为新请求从 `START` → `preflight` 重来。

没改变事实条件，就复用事实；改变了事实条件，就重新开始

三条路对应三种事实处理：原样复用、局部重写、彻底重来

## 等待不是异常，而是一次主动暂停

`awaitUser` 无新意图时返回 `NodeResult.interrupt()`。异常像列车脱轨，中断像列车按时进站——一个要排错，另一个本就在运行计划里。

1

执行器收到中断做三件事

先合并当前节点刚产生的状态变化 →

记录停下的位置和业务状态 → 返回

`awaitingUser`。

2

快照 `GraphExecutionSnapshot`

至少含：停下的节点、当时的

`GraphState`、`schemaVersion`。

即“做到哪、已知什么、什么版本”。

3

快照只留服务端

客户端只看到等待状态和允许展示的方案，拿不到节点游标、内部状态、

工具原始结果或半成品。

快照是内部恢复机制，不是对外开放的数据接口

把等待当异常，重试器会在用户思考时反复生成；把等待当完成，下一条消息就不知从哪接

## 恢复的铁律 · 先写入新意图，再选下一条边

```
PlanningEngine#resume
```

```
resume(GraphExecutionSnapshot snapshot, UserIntent userIntent)
```

正确顺序 · 关键五步：

1. 加载服务端快照
2. 识别本轮用户意图
3. 把新意图写入 `GraphState`
4. 重新进入 `awaitUser`
5. 根据新状态选择 `END` 或 `generate`

像修改导航目的地——必须先把新目的地写进导航，再让它选路线。若先用旧状态选边，用户刚说的话就没参与控制流判断；新意图也必须真正写入 `GraphState`，否则重启后丢失。

重新进入 `awaitUser`：有尚未处理的新意图就继续选路；没有新意图才再次中断

## 双环 · 机器内环求合格，用户外环求合意

机器内环 · 求合格

`generate` → `validate` → 返工 → `generate`

解决 " 方案是否合格 "。

`validate` 发现问题且还有轮次，  
就让 `generate` 自动修改再验收。

消耗：模型返工轮次、`maxRounds`。

用户外环 · 求合意

`generate` → `validate` → `awaitUser` → 续修

解决 " 方案是否合意 "。

机器判定可交付后，  
用户仍可提小范围修改；  
只有用户满意才进 `END`。

消耗：用户修订次数、会话时间。

两个环共享一份 `GraphState`，但绝不共用预算

机器自动返工几次，不能扣掉用户修改机会；用户思考十分钟，也不能算进模型执行超时

## Prompt W · 只增加一个等待点，不重写已有能力

### Prompt W (节选)

基于当前 7 节点 14 边执行图和 PlanningSession 持久化做增量：

1. 增加唯一等待节点 `awaitUser`，最终保持 8 节点、16 边；
2. `validate` 通过后进入 `awaitUser`，不再直接到 `END`；
3. `awaitUser` 无新意图时返回 `interrupt`；先合并增量，再保存含当前节点、`GraphState`、`schemaVersion` 的服务端快照，返回 `awaitingUser`；
4. API 只返回等待状态和允许展示的方案，不暴露快照 / 游标 / 半成品；
5. `resume` 先把新意图写入 `GraphState`，再重进 `awaitUser` 选边：满意去 `END`，小改去 `generate` 并复用已有事实；
6. 日期或目的地改变时结束当前链路，作为新请求从 `START-preflight`；
7. 机器返工轮次与用户修订次数分开记，等待不计入模型执行超时。

先给 " 首次等待、满意结束、小修改续修、需求变化重开 " 四条轨迹，确认后实现。

## 现场验证 · 等得住、接得上、不会重复查

1

提交 "杭州三天"

完成四项事实查询、生成、验收后停在 *awaitUser*；客户端显示等待，服务端存快照——不是报错，也不是 *END*

2

输入 "满意"

*awaitUser* → *END*，四项事实查询次数都不增加；走到这里这轮会话才真正完成

3

输入 "第二天轻松一点"

*awaitUser* → *generate* → *validate* → *awaitUser*；四个事实节点不再执行，说明复用了已有事实

4

等待中重启服务再发决定

仍从服务端快照恢复，结果与重启前一致；只能同进程恢复就不是可靠快照

能恢复却又把航班酒店重查一遍，不算接上——真正的恢复是带着原事实从用户决定处继续

两条反向检查：改日期 / 目的地必须按新请求重新预检查询；API 响应里不应出现快照与内部状态



## 本节小结

把 "等待用户" 从图外正式搬进图里：走到 `awaitUser` 就保存服务端快照并退出，  
新消息到达先把用户意图写进 `GraphState`，再重进 `awaitUser` 选边。图长成 8 节点 16 边。

### 回到主线

四条边界：中断不是异常；快照只留服务端；恢复先写状态再选边；内环求合格、外环求合意，共享状态不共用预算。

### 你带走的能力

会把 "等待" 做成一等公民的控制节点：能停下、能保存现场、能跨进程恢复，又不搞过度设计。

看得见的控制流——图和并行的可视化

页面上画的图，得就是正在跑的那张图

同源展示原则：页面上的节点和箭头，不许前端另抄一份。后端导出执行器正在使用的图声明，前端只负责摆位置、上颜色——执行、测试、展示共同读取同一张图。

#### 页面读取规则

# 页面将如何读图 / 着色（静态走读，不在本页操作）

运行时图声明 → 节点 / 边 / Join → 页面布局

preflight 完成 → 显示 done

四路 TOOL\_STARTED 可在并行启动时出现 → 显示 running

整组收口 → 按完成记录更新 done / degraded

四路都有交代后 → 再点亮 generate → validate → awaitUser

像机场大屏直接读真实航班系统，而不是另抄一张时刻表——少抄一份，就少一个骗人的机会

# 第一笔账 · 顺序：别强求四路按编号排队回来

压成八个字就好记：组内无序，组间有序

1

四路之间 · 不讲先后

四个窗口同时办，谁先完成每次都不一样——测试不固定航班、酒店、景点、天气的完成次序

2

组和组之间 · 讲先后

整组一定在预检通过之后才开始、在开始生成之前全部有交代，这条边界不动

3

单看一路 · 事件分两段

`TOOL__STARTED` 可在并行启动时出现；完成或降级先记入整组，收口后再按完成顺序发送

并行采集，收口后串行播报结果：四路可以在不同线程执行，`TOOL__STARTED` 也可在启动阶段出现；但 `TOOL__COMPLETED` / `TOOL__DEGRADED` 不是某一路一完成就实时推送，而是在整组收口后读取完成记录，按完成顺序经同一条 `SSE` 通道逐条发送。排队的是结果播报，不是采集。

测试检查 "该发生的都发生了、该有的组间先后还在"——组内完成顺序可以变，约束不能变

## 第二笔账 · 喊停：取消键只答应做得到的那部分

1

协作式取消 · 多点检查

取消令牌不能只在老循环开头看一次：  
提交任务前、等待过程中、进入下一个节点前都要检查；观察到取消后停止继续推进，也不再启动 *generate*

2

底层请求 · 可能继续

已经发出的本地或远端调用不一定立刻停，远端甚至可能已经收到。页面只写 "取消已受理"，不写 "所有外部调用已停止"

3

证据边界 · 只说测过的

当前自动化覆盖的是任务仍阻塞时取消：整组不再合入、*generate* 不启动；完成与取消撞线，仍需额外竞态测试

像球赛终场哨：哨声不会让已经踢出的球消失；完成与取消同时撞线时，判定规则必须另做竞态验证

当前证据覆盖任务仍阻塞时的取消；不承诺所有取消竞态都不合入，完成与取消撞线仍需额外测试

## 第三笔账 · 人挤：窗口开多了，得算清资源账

资源边界

# 当前资源边界

```
looptrip.graph.max-concurrent=8  # 同时运行最多 8 个
queue-capacity=32                 # 排队最多 32 个
single-node-timeout=5s            # 单节点 5 秒超时
group-timeout=10s                 # 整组 10 秒到点收口
```

队列满 → 结构化降级；绝不进入无上限队列

线程池隔离 + 截止线： *planning-pool* 与 *tool-pool* 分开，外部接口卡住不拖死整体规划；排队任务同样受整组 10 秒 *deadline* 约束，轮到时已经过点就不再发外部请求。

降级消息要一路带到下游：航班 / 酒店缺失进入 *GUARDED*；景点 / 天气缺失可带明确警告交付

## Prompt X · 只做同源展示，不夹带别的改造

Prompt X

给当前 LoopTrip 做一个讲师本地演示页，不改 8 节点 16 边，也不新增业务能力。

后端增加 `/api/graph`，直接从运行时 `GraphDefinition` 导出节点、边和 `Join`；

前端不要手写第二份图，只按节点 ID 布局，并用现有 SSE 更新

`idle`、`running`、`done`、`degraded`、`waiting` 状态。

先说明准备复用的接口和事件，再实现一条 " 杭州三天 " 完整演示：

`preflight` → 四事实节点 → `generate` → `validate` → `awaitUser`。

只做教学演示，不做监控平台，不换前端框架，也不修改事件名称。

下一页只跑一次主流程：S47 提交一次 " 杭州三天 "，观察四路运行与整组收口。边界场景不逐个现场重演，只核对三项自动化结果：降级、`GUARDED`、取消；本页只预告，不做实际页面操作。

## 效果演示 · 提交 " 杭州三天 " ，让图自己把话说完

唯一一次实际页面演示：速度不报固定数字，先看第三节脚本的串并行实测；再提交一次 " 杭州三天 " ，页面状态以现场实际为准。

### 看画面

主流程按实际状态点亮

*preflight* 先绿 → 四路进入

*running*；正常完成才绿、发生降级才橙。整组收口后再点亮

*generate*、*validate*，最后停在  
*awaitUser*。

### 跑脚本

课程快照给出结构与边界证据

脚本：*ch09-s5-graph-view.sh*

*businessNodes=8 • edges=16*

*queuedExternalCalls=0*

*CANCELLED • modelCalls=0*

*lateResultsMerged=false*

### 讲边界

快照事实与现场状态分开

脚本输出来自课程内置快照；降级、*GUARDED*、取消看自动化结果。页面不预设哪一路降级，也不预设终态，现场状态以实际为准。



## 本节小结

先让正在跑的图直接驱动页面：执行器跑哪张图，页面就画哪张图，测试也盯着同一张图。

再把并行多出来的三笔账一次结清：顺序、喊停、人挤。

### 三笔账怎么结的

顺序：组内无序、组间有序，整组收口后串行播报结果；

喊停：当前只验证任务仍阻塞时取消，完成与取消撞线

仍需竞态测试，在途请求也可能继续；人挤：线程池隔

开、队列封顶、到点收口。

### 你带走的能力

会让图声明成为执行、测试、展示的共同资产，也会为

并行系统诚实处理顺序、取消证据边界和资源上限。

本章小结：从 Loop Engineer 到 Graph Engineer

加的是 "能拆结构" 这档新本事，不是又多学一门技术

产品能做到哪一步 · 六档

能跑 → 能交付 → 能托管 → 能对话 → 能进化

第六档：能拆结构

本章加上的，正是这一档

手里有几样工具 · 就五样

*Prompt · Context · Harness · Loop · Graph*

*Graph* 只在最上层把前四样组织起来

没有凭空多出一样新工具

产品能力多了一档 "能拆结构"，工具箱还是那五样——两把尺子各量各的，别相加

就像施工队还是那几个工种，只是这回学会了看着整张施工图同时开工

## 真正带走的，是一组说得清 "为什么" 的选择

*Graph* 没有把前四样工具换掉，而是把它们装进一个能审查、能断言、能恢复的结构里。所以这章结束，手上拿到的绝不只是一个 *LightGraph* 类，更值钱的是一串能讲清 "为什么这么做" 的选择——包括为什么内环求合格、外环求合意，两个环共用同一份 *GraphState* 却绝不共用一笔预算：

1

为什么先画依赖再谈并行

依赖写成边，执行器才看得见哪几路  
能一起跑

2

为什么返工只回 generate

不重新查一遍事实；必需数据缺了走  
*GUARDED*，不让模型猜着补

3

为什么喊停只答应一半

事件只保证组内无序、组间有序；取消只保证 "晚到结果不采用"

从 "我能让它转起来"，到 "我能说得清它为什么这样转"——这就是 *Loop Engineer* 走到 *Graph Engineer* 的分水岭

## 后面两章 · 把技术翻译成职业语言

10

职业转译 (约 30 分钟)

不再增加技术模块。把这一路的工程选择翻译成岗位语言：

*Agent* 应用开发工程师需要哪些能力、面试怎样讲架构取舍、

简历怎样把改造成可信证据、被追问细节时怎样接住

11

全景总结 (约 15 分钟)

为整门课程做一次全景总结，正式结课

第九章的技术活到此全部干完：一条 Loop 被展开成能并行、能暂停、能恢复、能画出来、能被测试盯住的 Graph

下一章带着这些收获，接着往职业语言上走——*Agent* 面试辅导